

Data Structure and Algorithm, Spring 2026

Homework 3

Due: 13:00:00, Tuesday, May 19, 2026

TA E-mail: dsa_ta@csie.ntu.edu.tw

Rules and Instructions

- Any form of cheating, lying, or plagiarism will not be tolerated. Students can get zero scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.
- Discussions on course materials and homework solutions are encouraged. But you should write the final solutions alone and understand them fully. Books, notes, and Internet resources (including chatGPT) can be consulted, but not copied from.
- Since everyone needs to write the final solutions alone, there is **absolutely no need to lend your homework solutions and/or source codes to your classmates at any time**. In order to maximize the level of fairness in this class, lending and borrowing homework solutions are both regarded as dishonest behaviors and will be punished according to the honesty policy.
- We will detect and penalize plagiarism with the criteria of **line-by-line similarity** in any part of your code/writing, when compared with the code/writing from any student or any external resource. Exceptions will be given to those showing **time-stamped records** that the solution comes from a meaningful interactive discussion with human and/or AI tools, **rather than copying from** others.
- In Homework 3, the problem set contains 4 problems. The set is divided into two parts, the non-programming part (Problems 1, 2) and the programming part (Problems 3, 4).

- For your solutions in the non-programming part, unless otherwise noted, there are two common requirements:
 - Your answer should be as simple as possible. For instance, $\Theta(\frac{1}{n} + \lg(n^{541}) + 8763n^2)$ should be written as $\Theta(n^2)$.
 - Your answer should also come with a *brief* explanation. The word *brief* means “very short.” An example of a *brief* explanation is as follows.

Sample problem: Write down the time complexity for this algorithm using the Θ notation along with an expression of n .

HELLO-WORLD(n)

```
1  for  $i = 1 \dots 2n$ 
2      for  $j = 1 \dots 2n$ 
3          print("Hello world")
```

Sample solution: The iteration counts of the two loops are $2n$ and $2n$ respectively, so there are $4n^2$ iterations. Hence, the time complexity is $\Theta(n^2)$.

□

- For problems in the non-programming part, you should combine your solutions in *one* PDF file. Your file should generally be legible. Your solution must be as simple as possible. At the TAs' discretion, solutions which are too complicated can be penalized or even regarded as incorrect. If you would like to use any theorem which is not mentioned in the classes, please include its proof in your solution.
- The PDF file for the non-programming part should be submitted to Gradescope as instructed, and you should use Gradescope to tag the pages that correspond to each sub-problem to facilitate the TAs' grading. **A 10% penalty applies if no pages are tagged, and a 5% penalty if at least one (but not all) correct pages are tagged.**

- For problems in the programming part, you should write your code in C programming language, and then submit the code via the *DSA Judge+*: <https://dsa.csie.ntu.edu.tw/>. Each day, you can submit up to 10 times for each problem. The judge system will compile your code with

```
gcc [id].c -O2 -lm -std=c17
```

- For debugging in the programming part, we strongly suggest you produce your own test-cases with programs and/or use tools like `gdb` or compile with flag `-fsanitize=address` to help you solve issues like illegal memory usage. For `gdb`, you are strongly encouraged to use compile flag `-g` to preserve symbols after compiling.

Note: For students who use IDE (VScode, DevC++...) you can modify the compile command in the settings/preference section. Below are some examples:

```
- gcc [id].c -O2 -lm -std=c17 -fsanitize=address
                                # Works on Unix-like OS, but possibly not M1/M2
- gcc [id].c -O2 -lm -std=c17 -g # use it with gdb
```

- The score of the part that is submitted after the deadline will get some penalties according to the following rule:

$$Late\ Score = \max \left(\left(\frac{5 \times 86400 - Delay\ Time(sec.)}{5 \times 86400} \right) \times Original\ Score, 0 \right)$$

As mentioned at the beginning of the class, you have **eight gold medals** this semester, each of which can be used to extend the deadline of **one** programming problem by **24 hours**. You can choose how to allocate the medals around the end of the semester.

- We use the gender-neutral pronouns *ze/zir* throughout this problem set.
- If you have questions about this homework, please go to the Discord channel and discuss the issues with your classmates. This is *strongly preferred* because it will provide everyone with a more interactive learning experience. If you really need email clarifications, you are encouraged to include the following tag in the subject of your email:
 - Please include the following tag in the subject of your email: "[HW3.Px]", specifying the problem where you have questions. For example, "[HW3.P2] Is k in subproblem 5 an integer?". Adding the tag allows the TAs to track the status of each email and to provide faster responses to you.

- If you want to provide your code segments to us as part of your question, please upload it to [Gist](#) or similar platforms and provide the link. Please remember to protect your uploaded materials with proper permission settings. Screenshots or code segments directly included in the email are discouraged and may not be reviewed.

Problem 1 - Dominating-String Accomplishment (100 pts)

Before proceeding to the problem set, we establish the following notations and definitions to ensure clarity in the following tasks:

- The character set $\Sigma = \{c_1, c_2, \dots, c_{|\Sigma|}\}$ is the set of characters that can be used to form any string in this problem.
- You can perform these operations in $O(1)$ time and space: (a) comparison of two characters; or (b) obtain the index i of a character $c_i \in \Sigma$, denoted $\text{idx}_\Sigma(c_i) = i$. In addition, you can iterate over all characters in $\Sigma = \{c_1, c_2, \dots, c_{|\Sigma|}\}$ in $O(|\Sigma|)$ time.
- A string S is an array of characters drawn from Σ with a finite length. Let $|S|$ denote the length of the string, and $S[i]$ denote the i -th character (1-indexed) in S .
- A substring is a contiguous sequence of characters within a string. We use $S[l:r]$ to denote the substring $S[l]S[l+1] \dots S[r-1]S[r]$.
- A prefix is a substring $S[l:r]$ where $l = 1$; a suffix is a substring $S[l:r]$ where $r = |S|$.
- Let S^c denote c repetitions of string S , i.e., $S^c = \underbrace{SS \dots S}_{c \text{ copies}}$.

The KMP prefix function introduced in the lecture is provided below. You can use it in your solution when applicable. The prefix function requires a string P as the parameter, and will return an array $\pi[1:|P|]$, where $\pi[i]$ is the largest $k < i$ such that $P[1:k] = P[i-k+1:i]$, $i = 1, 2, \dots, |P|$. The pseudo code runs in $O(|P|)$ time and space.

Pseudo Code

```
COMPUTE-PREFIX-FUNCTION( $P$ )
1   $m = |P|$ 
2   $\pi[1:m] = \text{new array}$ 
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11 return  $\pi$ 
```

1. (20 pts) Repetitive string matching. In this problem, you are given a string T of length n and a string P of length m , along with a positive integer c . The goal is to count the number of occurrences of P in the repetitive string T^c . Note that $|T| \geq |P|$ no longer needs to hold, but $|T^c| \geq |P|$ does.

A student proposed the following pseudo code (REPETITIVE-STRING-MATCHING()), a variant of the KMP algorithm, to solve this problem. The student claims that its running time is $O(nm + c)$. The analysis is incorrect and the judge system also reported a TLE. On the other hand, the algorithm does produce correct results.

Please provide a counterexample to show that the running time is not $O(nm + c)$. In addition, please derive the correct time complexity of REPETITIVE-STRING-MATCHING().

Pseudo Code

```
REPETITIVE-STRING-MATCHING( $T, P, c$ )
1   $n = |T|$ 
2   $m = |P|$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $f[0:m-1] = \text{new array}$ 
5   $g[0:m-1] = \text{new array}$ 
6  for  $j = 0$  to  $m - 1$ 
7       $q = j$ 
8       $cnt = 0$ 
9      for  $i = 1$  to  $n$ 
10         while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
11              $q = \pi[q]$ 
12         if  $P[q + 1] == T[i]$ 
13              $q = q + 1$ 
14         if  $q == m$ 
15              $cnt = cnt + 1$ 
16              $q = \pi[q]$ 
17          $f[j] = q$ 
18          $g[j] = cnt$ 
19   $q = 0$ 
20   $cnt = 0$ 
21  for  $i = 1$  to  $c$ 
22       $cnt = cnt + g[q]$ 
23       $q = f[q]$ 
24  return  $cnt$ 
```

Below are the explanations provided by the student along with the pseudo code. These may help you understand the algorithm.

- By observing how the original KMP string matching algorithm finds P from T^c , I found an interesting phenomenon. That is, due to the periodicity of T^c , the matching procedure is also periodic and predictable. For example, when given $T = abbc$, $P = bcabb$, $c = 4$, the first few iterations of the matching produce the following q values:
- In the example above, the **pattern pointer** starts at $q = 0$. Upon completing the

T^4		a	b	b	c	a	b	b	c	a	b	b	c	a	b	b	c	...
pos		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	...
r		1				2				3				4				...
q	0	0	1	1	2	3	4	1	2	3	4	1	2	3	4	1	2	

first copy of T (i.e., $r = 1$, $pos = 4$), the pattern pointer lands at $q = 2$. Upon completing the second copy (i.e., $r = 2$, $pos = 4$), it again lands at $q = 2$, and one occurrence of P was found during this copy.

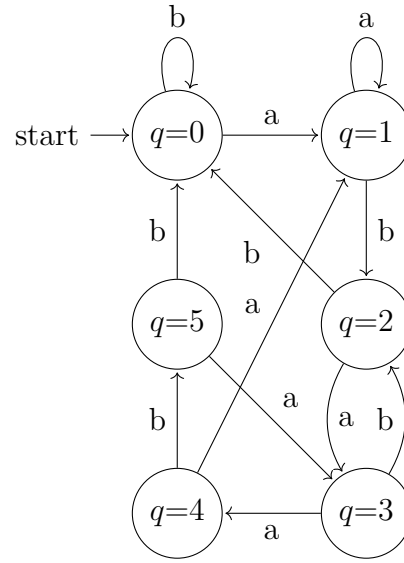
- I then observed that when KMP processes a copy of T in T^c , the ending value of the pattern pointer q and the number of matches found depend only on the value of q at the start of that copy — not on which copy it is. Therefore, we can precompute two arrays: $f[j]$ stores the ending value of q after processing one T starting from $q = j$, and $g[j]$ stores the number of matches found. These arrays are built by **running m separate KMP simulations on T** , one for each starting value $j \in \{0, 1, \dots, m-1\}$.

2. (30 pts) The repetitive string matching problem in Problem 1.1 can be solved more efficiently using the *prefix automaton* — an extension of the KMP prefix function. Recall that the prefix function only tells us where to fall back **on a mismatch**. Specifically, the prefix automaton function $\mathfrak{A}(q, c)$ outputs the next pattern pointer position given the current pattern pointer position q and the next input character $c \in \Sigma$. This eliminates the need to follow failure links with multiple steps at runtime, and instead performs a simple constant-time table lookup.

Below is an example of the prefix automaton for $P = abaab$ with $\Sigma = \{a, b\}$.

Given $\mathfrak{A}$, string matching with pattern P reduces to a simple left-to-right scan: start at $q = 0$ and for each character c of the text, transition to $\mathfrak{A}(q, c)$. A match is found whenever q reaches m . The automaton above visualizes the same transitions.

$\mathfrak{A}(q, c)$		
$q \backslash c$	a	b
0	1	0
1	1	2
2	3	0
3	4	2
4	1	5
5	3	0



Pseudo Code

COUNT-OCCURRENCE(T, P)

```

1   $n = |T|$ 
2   $m = |P|$ 
3   $\mathfrak{A} = \text{COMPUTE-PREFIX-AUTOMATON}(P)$ 
4   $q = 0$ 
5   $cnt = 0$ 
6  for  $i = 1$  to  $n$ 
7       $q = \mathfrak{A}(q, \text{idx}_{\Sigma}(T[i]))$ 
8      if  $q == m$ 
9           $cnt = cnt + 1$ 
10 return  $cnt$ 

```

- (a) (15 pts) After identifying the inefficiency in zir solution to Problem 1.1, the student decided to use the prefix automaton for a more efficient approach. However, ze was interrupted to prepare for the Earth Game, leaving the implementation unfinished. Below is the student's partial implementation for constructing $\mathfrak{A}$ from P . Complete the remaining lines in no more than 15 lines of pseudo code, and provide a brief explanation of your design in no more than 5 lines. The algorithm should run in $O(m|\Sigma|)$ time and space.

Pseudo Code

```

COMPUTE-PREFIX-AUTOMATON( $P$ )
1   $m = |P|$ 
2   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
3   $\mathfrak{A} = \text{2D array with } (m+1) \times |\Sigma| \text{ entries}$ 
4   $// q \in \{0, \dots, m\}; c \in \{1, \dots, |\Sigma|\}$ 
5   $// \text{TODO: calculate all values in } \mathfrak{A}$ 

```

- (b) (15 pts) Using COMPUTE-PREFIX-AUTOMATON from Problem 1.2(a), provide an algorithm to solve Problem 1.1 in $O(m|\Sigma| + mn + c)$ time and $O(m|\Sigma|)$ space. Write down your algorithm in no more than 30 lines of pseudo code, then briefly explain your design and argue its correctness. You may call COMPUTE-PREFIX-FUNCTION and COMPUTE-PREFIX-AUTOMATON as subroutines, but account for their time complexities in your analysis.

***Hint:** As mentioned previously, the student's algorithm in Problem 1.1 produces correct results but has suboptimal time complexity. The prefix automaton is the key to optimizing the running time.*

3. (25 pts) We define a new *anagram* string matching operator $\simeq$ as follows. For two strings S_1 and S_2 of the same length, i.e., $|S_1| = |S_2| = n$, we have $S_1 \simeq S_2$ if and only if there exists a permutation $(p_1, p_2, \dots, p_n)$ of $(1, 2, \dots, n)$ such that $S_1[i] = S_2[p_i]$ for $i = 1, 2, \dots, n$.

For example, let $D_1 = dsdsa$ and $D_2 = assdd$. We have $D_1 \simeq D_2$ since $(p_1, p_2, p_3, p_4, p_5) = (4, 2, 5, 3, 1)$ satisfies $D_1[i] = D_2[p_i]$ for $i = 1, 2, \dots, n$.

We can now formulate a new *anagram* string matching problem. Given a string T and a string P , instead of using exact match of strings, i.e., the $=$ operator, we use the anagram operator $\simeq$ to compare the substrings in T and P . Let $n = |T|$ and $m = |P|$. The goal remains to find all valid shifts $0 \leq s \leq n - m$ in T such that $T[s+1:s+m] \simeq P$.

Derive an algorithm to solve the problem in $O((n+m) \log |\Sigma|)$ time and $O(1)$ extra space **with high probability**. Write down the pseudo code of your algorithm in at most 40 lines, explain why the code is correct, and show that it satisfies the time and space complexity requirements.

For the definition of “**with high probability**”:

You are expected to design a hashing technique that maps each string to an integer, similar to the one used in the Rabin-Karp string matching algorithm,

and treat two strings as having the same property if their hash values are equal **without checking character by character** (i.e., assuming no spurious hits). However, you must ensure that **different strings produce different hash values** when the hash range is unbounded — under a sufficiently powerful computation model that supports arbitrary-precision arithmetic, the algorithm becomes deterministic.

You may assume that exponentiation, i.e., computing x^y , takes $O(\log y)$ time and $O(1)$ space for non-negative integers x and y using *exponentiation by squaring*, and that each basic arithmetic operation ($+$, $-$, $\times$, $/$, and mod) on two integers takes $O(1)$ time and space.

4. (25 pts) Given a string S of length n , provide an algorithm that *counts the number* of integer pairs (l, r) with $1 \leq l \leq r \leq n$ such that $S[l:r]$ is **both a prefix and a suffix** of S .

Let $k = r - l + 1$. For $S[l:r]$ to be both a prefix and a suffix of S , two conditions need to be satisfied:

1. $S[1:k] = S[l:r]$; and
2. $S[1:k] = S[n - k + 1:n]$.

Try to use prefix function to recursively build a table to count the number of satisfied pairs. Your algorithm should run in $O(n)$ time and $O(n)$ space.

Write down the pseudo code of your algorithm in at most 40 lines, explain why it is correct, and demonstrate that it satisfies the time and space complexity requirements.

Problem 2 - Diverse Sorting Algorithm (100 pts)

1. (15 pts) Assume that an online system supports sorting a sequence of n integers. After investigation, you discover that it uses the following QUICK-SORT algorithm, a variant of the Lomuto quicksort. Here A is the 1-indexed array being sorted, L and R are the left and right boundaries of the current subarray (i.e., QUICK-SORT operates on $A[L:R-1]$), and $\text{SWAP}(A[i], A[j])$ exchanges the values of $A[i]$ and $A[j]$. The variable cnt counts the number of element comparisons $A[i] < A[L]$ performed in the current call, and $\text{QUICK-SORT-COUNT}(A)$ returns the total count accumulated across all recursive calls:

QUICK-SORT(A, L, R)

```
1  if  $R - L \leq 1$ 
2      return 0
3  SWAP( $A[\lfloor \frac{L+R}{2} \rfloor], A[L]$ )
4   $i = L + 1$ 
5   $p = L + 1$ 
6   $cnt = 0$ 
7  while  $i < R$ 
8       $cnt = cnt + 1$ 
9      if  $A[i] < A[L]$ 
10         SWAP( $A[i], A[p]$ )
11          $p = p + 1$ 
12      $i = i + 1$ 
13  SWAP( $A[L], A[p - 1]$ )
14  return  $cnt + \text{QUICK-SORT}(A, L, p - 1) + \text{QUICK-SORT}(A, p, R)$ 
```

QUICK-SORT-COUNT(A)

```
1  return QUICK-SORT( $A, 1, A.size + 1$ )
```

Construct a permutation of $[1, 2, \dots, 20]$ stored in array $A = [A_1, A_2, \dots, A_{20}]$ such that $\text{QUICK-SORT-COUNT}(A)$ returns a value of at least 150. Report your return value. You do not need to describe your construction process.

In the real world, this is called the [algorithmic complexity attack](#). If an input to deterministically induce worst-case running time can be constructed, then it can be used to launch an attack that consumes the system's resources, effectively a denial-of-service attack.

2. (15 pts) Let A be an array of length n , and let x be a split point. Intuitively, elements on the left side of the split point should be no larger than elements on the right side. A pair

that crosses the split point and violates this order can therefore be viewed as a partial inverse pair.

More formally, given positive integers k and x with $1 \leq x < n$, a **partial inverse pair** with respect to x and k is a pair (i, j) such that

$$i \leq x < j \quad \text{and} \quad A[i] > k \cdot A[j].$$

Let P denote the set of all partial inverse pairs:

$$P = \{(i, j) \mid i \leq x < j \text{ and } A[i] > k \cdot A[j]\}.$$

Assume that both subarrays $A[1:x]$ and $A[x+1:n]$ are sorted in nondecreasing order. Devise an algorithm to count $|P|$ in $O(n)$ time using $O(1)$ extra space. Write the pseudo code in at most 30 lines, and briefly explain why it is correct and satisfies the time and space complexity requirements.

3. (20 pts) Given an array B of length m and a positive integer t , define the set of **super inverse pairs** as

$$S = \{(i, j) \mid i < j \text{ and } B[i] > t \cdot B[j]\}.$$

Devise an algorithm to count $|S|$ in $O(m \log m)$ time using $O(m \log m)$ extra space. Write the pseudo code in at most 40 lines.

You may find it useful to build on the **partial inverse pairs** algorithm from Problem 2.2. You may call `PARTIAL-INVERSE-PAIRS( $A, k, x$ )` directly in your pseudo code. For an array A of length n , `PARTIAL-INVERSE-PAIRS( $A, k, x$ )` correctly counts and returns $|P|$ in $O(n)$ time using $O(1)$ extra space, provided that both subarrays $A[1:x]$ and $A[x+1:n]$ are sorted in nondecreasing order. Note that this function does **not** modify A . In Problem 2.2, `PARTIAL-INVERSE-PAIRS` is called on the full array $A[1:n]$. In Problem 2.3, you may adapt it to operate on a subarray $B[\text{low}:\text{high}]$ by passing the appropriate range parameters.

Briefly explain why your pseudo code is correct and satisfies the time and space complexity requirements.

Note: When $t = 1$, this problem reduces to counting the number of swaps performed by BUBBLE-SORT. This shows how to count inversions in $O(m \log m)$ time, compared with the $O(m^2)$ time required by classic BUBBLE-SORT.

4. (20 pts) Given a set $\mathcal{A} = \{A_1, A_2, \dots, A_n\}$ of n strings where $\sum_{i=1}^n |A_i| = m$, and assume that comparing two characters takes $O(1)$ time.

Define **Lexicographical Order** as follows: for any two strings S and T , $S < T$ if and only if:

1. There exists an index $k \leq \min(|S|, |T|)$ such that $S[i] = T[i]$ for all $i < k$, and $S[k] < T[k]$; or
2. $S[i] = T[i]$ for all $i \leq \min(|S|, |T|)$ (one string is a prefix of the other) and $|S| < |T|$.

Note that comparing two strings S_1 and S_2 takes $O(|S_1| + |S_2|)$ time in the worst case. Devise an algorithm and write down the pseudo code within 40 lines to sort $A_1, A_2, \dots, A_n$ in **Lexicographical Order** in $O(m \log n)$ time and $O(m+n)$ extra space. Briefly explain why your pseudo code is correct and meets the required time and space complexity.

5. (30 pts) The following problem is presented in two equivalent versions. You may choose whichever version(s) you find easier to work with.

Story version: The Diverse Sorting Administration is holding a competition, and our DSA class must select students to form a team.

There are n students $T_1, T_2, \dots, T_n$ in the class. Each student T_i has two positive attributes: $T_i.speed$ and $T_i.score$, for $1 \leq i \leq n$.

Given an integer k with $1 \leq k \leq n$, select a set of indices $S \subseteq \{1, 2, \dots, n\}$ with $|S| = k$ to maximize the team performance: $P(S) = \left(\sum_{x \in S} T_x.score \right) \cdot \min_{x \in S} (T_x.speed)$.

Mathematical version: Given n objects $T_1, T_2, \dots, T_n$, each with two positive attributes: $T_i.speed$ and $T_i.score$, for $1 \leq i \leq n$, and an integer k with $1 \leq k \leq n$. For a set $S \subseteq \{1, 2, \dots, n\}$ with $|S| = k$, define $P(S) = \left(\sum_{x \in S} T_x.score \right) \cdot \min_{x \in S} (T_x.speed)$. Find the maximum $P(S)$ over all such sets S .

Devise an algorithm and write down the pseudo code within 30 lines to compute the maximum possible $P(S)$ in $O(n \log n)$ time and $O(n)$ extra space. Briefly explain why your pseudo code is correct and meets the required time and space complexity.

Problem 3 - Dubious System Authentication (100 pts)

Problem Description

We have learned some string-matching algorithms such as Rabin-Karp and KMP in class, which efficiently find substrings within larger strings. However, these algorithms are not designed for scenarios where we need to deal with multiple strings. For example, when implementing a user account system, it becomes crucial to find out whether the newly registered username is unique among existing usernames efficiently.

In this problem, we suggest you learn a new data structure called **Trie** to address prefix searching more efficiently. The following description of **Trie** might be helpful for you.

Reference Reading

Definitions. Trie is a specialized tree-based data structure used for storing and retrieving strings, typically in applications where prefix-based operations are crucial. Tries are particularly efficient for tasks such as autocomplete systems, spell checkers, or dictionary management. Let T be a Trie storing a *set* of strings $S = \{s_1, s_2, \dots, s_n\}$, where each string is composed of characters from a fixed alphabet Σ .

- Each edge represents a character in Σ .
- The root node represents a null string.
- Each node in the Trie represents a prefix of some strings in S . The prefix string can be obtained by concatenating all characters represented by the *edges* in the path from the root to the node.
- Each string $s \in S$ can be represented by a leaf node in the Trie. Similarly, concatenating the characters represented by the edges forming the path from the root to the leaf node produces the string s .

Figure 3.1 presents an example of Trie. Note that it branches at the common prefix of those in S , i.e., “ca”.

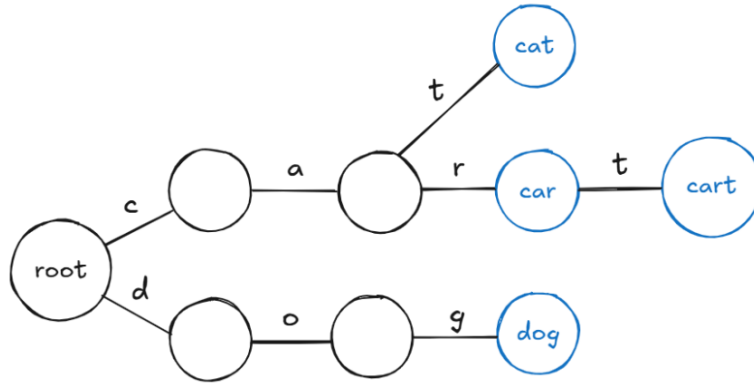


Figure 3.1: A Trie representing $S = \{\text{"cat"}, \text{"car"}, \text{"cart"}, \text{"dog"}\}$

Key properties of a Trie.

- **Prefix representation:** Every prefix of a string $s \in S$ can be represented by a node in T .
- **Search efficiency:** Finding a string or prefix of length m in a Trie takes $O(m)$ time, regardless of the number of strings stored in the Trie.
- **Space efficiency:** Common prefixes among the strings in S are stored only once.

Implementation. A Trie is constructed by inserting a string one character at a time. Each node stores the pointers to the child nodes, each of which corresponds to adding one more character drawn from Σ . In addition, a boolean flag is stored at each node, indicating whether the current node corresponds to a string $s \in S$. There are many ways to implement the child node pointers (e.g., a fixed-size array indexed by character, a hash map, or a linked list of existing children), each with different time and space trade-offs. Think carefully about which implementation best suits the constraints of this problem.

To *insert* a string s into Trie T :

- (1) Start from the root node.
- (2) For each character c in s : If c does not exist as a child of the current node, create a new node for c .
- (3) Move to the child node corresponding to c .
- (4) After processing all characters, mark the final node to indicate that the node corresponds to a string in S .

To *search* for a string s in the Trie:

- (1) Start from the root node.
- (2) Take the next character c in s . Traverse to the child node corresponding to c . If c cannot be found on an edge from the current node, stop the current search and return “not found.”

- (3) Repeat the previous step until all characters in s are processed. If all characters are found and the final node is marked as the end of a string, s exists in the Trie.

In this problem, you are tasked with building a simplified User Account Management System supporting three operations: registering a new user, logging in, and deleting an account. Both usernames and passwords consist of characters from $\Sigma = \{a, b, \dots, z, A, B, \dots, Z, 0, 1, \dots, 9\}$. To manage these operations efficiently, you must implement the following:

- **Username Management:** All usernames are stored in a Trie. Each node in the Trie represents a character. You must assign a unique **Serial Number** to *each node in the order they are created*. Specifically, the root node is always created first and assigned ID 0. For example, when inserting “NTU”, if ‘N’ is not a child of the root, create a new node with ID 1, then ‘T’ gets ID 2, and ‘U’ gets ID 3. Note that the serial number will not be recycled in the case that a username is deleted. That is, if any nodes are deleted, their serial numbers are retired. The serial number will always be the last used serial number plus one when a new node is inserted.
- **Password Verification:** To adhere to security best practices, passwords must never be stored in plaintext. Instead, the system stores a hash value to serve as an authentication token. While production-grade systems typically employ cryptographic algorithms like SHA-256 to virtually eliminate collision risks, such algorithms fall outside the scope of this course. Therefore, this problem utilizes the Rolling Hash (the core of the Rabin-Karp algorithm) for password verification. To mitigate the collision probability inherent in a single hash, you are required to implement a **Doubly Rolling Hash**, ensuring a more robust authentication mechanism.

Specifically, the two hash functions are defined as follows for a password string S of length n , where M_i is the modulus and B_i is the base:

$$\begin{aligned}
 - M_1 &= 10^{16} + 61, B_1 = 67, H_1(S) = \sum_{i=1}^n idx(S[i]) \times B_1^{n-i} \pmod{M_1} \\
 - M_2 &= 10^{16} + 99, B_2 = 71, H_2(S) = \sum_{i=1}^n idx(S[i]) \times B_2^{n-i} \pmod{M_2}
 \end{aligned}$$

Here, $idx(c)$ is the index of c in Σ : ‘a’ = 1, ‘b’ = 2, ..., ‘z’ = 26, ‘A’ = 27, ‘B’ = 28, ..., ‘Z’ = 52, ‘0’ = 53, ‘1’ = 54, ..., ‘9’ = 62.

There are three operations in this problem:

- **Register:** The system will receive a username and password. If the username is not already present in the Trie, insert it and store the two password hash values. If the username is already present, see below in the Output section about how to handle the case.
- **Login:** The system will receive a username and password. If the username exists, the

system compares the calculated hash values of the input password with the stored records to determine if the login is successful. If the username is not present, see below in the Output section about how to handle the case.

- **Delete:** The system will receive a username only. If the user exists, the system removes the user's account. To maintain memory efficiency, you need to free the redundant nodes in the Trie. Specifically, a node will be freed if and only if there no longer exist any strings in the Trie requiring that node to represent. **The root node of the Trie is a special case, it will never be freed.** If the username is not present, see below in the Output section about how to handle the case.

Input

The first line contains one integer Q , representing the total number of operations. Each of the next Q lines is given in one of the following formats:

- **Register** `<user>` `<pass>`: Registers a new user with the given username `<user>` and password `<pass>`.
- **Login** `<user>` `<pass>`: Authenticates the user by checking if the username `<user>` exists and the password `<pass>` hashes match.
- **Delete** `<user>`: Deletes the user account if `<user>` exists, followed by freeing the redundant nodes as described.

Output

Let `<hv1>` denote $H_1(\text{<pass>})$ and `<hv2>` denote $H_2(\text{<pass>})$. Note that `<pass>` is the parameter of each operation, which might not be the correct password.

The output of each operation is composed of traversal output and operation output.

Traversal Output: For each operation, perform exactly one traversal starting from the root node (ID 0) and following the characters of `<user>` in order.

- For **Register**: If a child node does not exist, create it and continue the traversal.
- For **Login** and **Delete**: If a child node does not exist, stop the traversal immediately.

During traversal, output the serial numbers of all visited nodes in order on a single line.

- If traversal stops early, print only the nodes visited so far.
- Newly created nodes during **Register** should also be printed.

Operation Output: After the traversal output, determine and print the result of the operation as follows:

- **Register:** Output `Username already exists` if the username is already registered. Otherwise, output `Successfully registered <hv1> <hv2>`

- **Login:** Output `User does not exist` if the username does not exist. Otherwise, output `Incorrect password <hv1> <hv2>` if the password is wrong. Otherwise, output `Successfully logged in`
- **Delete:** Output `User does not exist` if the username does not exist. Otherwise, output `Successfully deleted`

The operation output must be generated after complete traversal output.

Constraints

- $1 \leq Q \leq 10^4$
- The length of username and password does not exceed 100.
- The character set $\Sigma = \{ a, b, \dots, z, A, B, \dots, Z, 0, 1, \dots, 9 \}$.

Subtasks

Subtask 1 (35 pts)

- Only includes **Register** operation.

Subtask 2 (15 pts)

- Only includes **Register** and **Login** operations.

Subtask 3 (50 pts)

- Includes all operations.

Sample Testcases

Sample Input 1

```
5
Register AAA aaa
Register BBB bbb
Register AAAB aaab
Register AAA aaa
Register AA aa
```

Sample Output 1

```
0 1 2 3
Successfully registered 4557 5113
0 4 5 6
Successfully registered 9114 10226
0 1 2 3 7
Successfully registered 305321 363025
0 1 2 3
Username already exists
0 1 2
Successfully registered 68 72
```

Sample Input 2

```
5
Register AAA aaa
Login AAA aaa
Login AAA bbb
Login AAB aaa
Register AAA bbb
```

Sample Output 2

```
0 1 2 3
Successfully registered 4557 5113
0 1 2 3
Successfully logged in
0 1 2 3
Incorrect password 9114 10226
0 1 2
User does not exist
0 1 2 3
Username already exists
```

Sample Input 3

```
10
Register AAA aaa
Register AAAB aaab
Delete AAA
Login AAA aaa
Login AAAB aaab
Register AAA ccc
Login AAA aaa
Delete AAAB
Register ABC abc
Login ABC abc
```

Sample Output 3

```
0 1 2 3
Successfully registered 4557 5113
0 1 2 3 4
Successfully registered 305321 363025
0 1 2 3
Successfully deleted
0 1 2 3
User does not exist
0 1 2 3 4
Successfully logged in
0 1 2 3
Successfully registered 13671 15339
0 1 2 3
Incorrect password 4557 5113
0 1 2 3 4
Successfully deleted
0 1 5 6
Successfully registered 4626 5186
0 1 5 6
Successfully logged in
```

Problem 4 - Divine Soccer Academy (100 pts)

Problem Description

We will provide a story version followed by an equivalent mathematical version. Please feel free to decide which version(s) you want to read to solve this problem.

Story version: DSA (Divine Soccer Academy) hosts an annual one-on-one exhibition between two prodigies: ALICE, the dazzling attacker, and BOB, the unflinching gatekeeper. The pitch is unusual: it consists of N marked positions, labeled 1 through N , connected by M one-way passing lanes. Every position is the territory of exactly one of the two players — either an attacker's spot or a gatekeeper's spot.

A match begins with the ball placed on the starting position s , and it ends the instant the ball arrives at the goal position t . While the ball lies on a position v , only the outgoing lanes $v \rightarrow w$ are legal next moves. If v is an attacker's spot, ALICE picks one such lane and kicks the ball to w ; if v is a gatekeeper's spot, BOB picks one and deflects the ball to w . Each kick or deflection counts as one move. If the ball is on a position with no outgoing lanes, it is stuck there forever.

ALICE wants to drive the ball to t in as few moves as possible. BOB wants to delay the ball for as long as possible — forever, if he can manage it. Assuming both sides play optimally, determine how many moves it takes for the ball to reach t ; or report -1 if BOB has a strategy that prevents the ball from ever reaching t .

Mathematical version: You are given a directed graph $G = (V, E)$ with $|V| = N$ vertices labeled 1 through N and $|E| = M$ directed edges. The vertex set is partitioned as $V = V_A \cup V_B$, $V_A \cap V_B = \emptyset$. A token is initially placed on a starting vertex $s \in V$, and a target vertex $t \in V$ is given.

Two players, A and B , move the token along the directed edges as follows. At each step, let v be the current vertex of the token. If $v = t$, the game terminates immediately. Otherwise, exactly one player chooses one outgoing edge $\langle v, w \rangle \in E$ and moves the token along it: player A chooses if $v \in V_A$, and player B chooses if $v \in V_B$. Traversing each edge counts as one move. If $v \neq t$ has no outgoing edges, the game is stuck.

The two players have opposite goals. Define the game value $V^*(v)$ as the number of moves to reach t from v under optimal play by both players: player A wants to make $V^*(v)$ as small as possible, and player B wants to make it as large as possible (in particular, $+\infty$ if at all possible). Plainly, $V^*(t) = 0$. If the game is stuck at a vertex with no outgoing edges, the game value of that vertex is $+\infty$. More generally, the value is $+\infty$ on every vertex from which player A cannot avoid an infinite sequence of moves. Your task is to compute $V^*(s)$, or report -1 if $V^*(s) = +\infty$.

Input

The inputs are described with the mathematical version; the notation corresponds directly to the story version.

- The first line contains four space-separated integers N , M , s , and t , representing the number of vertices, the number of directed edges, the starting vertex, and the target vertex, respectively.
- The second line contains N space-separated integers $c_1, c_2, \dots, c_N \in \{0, 1\}$, where $c_i = 0$ indicates $i \in V_A$ (an attacker's spot) and $c_i = 1$ indicates $i \in V_B$ (a gatekeeper's spot).
- Each of the next M lines contains two space-separated integers u and v , representing a directed edge $\langle u, v \rangle$.

Output

Output a single integer on one line: the number of moves for the token to reach t starting from s under optimal play by both players (i.e., $V^*(s)$), or -1 if the token never reaches t under optimal play (i.e., $V^*(s) = +\infty$). If $s = t$, the answer is 0.

Constraints

- $1 \leq N \leq 10^6$
- $0 \leq M \leq 10^6$
- $1 \leq s, t \leq N$ (it is possible that $s = t$)
- $c_i \in \{0, 1\}$ for every $1 \leq i \leq N$
- $1 \leq u, v \leq N$
- The graph contains no parallel edges, but self-loops (edges of the form $\langle u, u \rangle$) are allowed

Subtasks

Subtask 1 (15 pts)

- $N \leq 10^3$ and $M \leq 10^3$.

Subtask 2 (15 pts)

- $c_i = 0$ for every $1 \leq i \leq N$.

That is, every vertex is an attacker's spot ($V_A = V$).

Subtask 3 (15 pts)

- $c_i = 1$ for every $1 \leq i \leq N$.

That is, every vertex is a gatekeeper's spot ($V_B = V$).

Subtask 4 (25 pts)

The graph G is acyclic (a DAG).

Subtask 5 (30 pts)

No additional constraints.

Sample Testcases

Sample Input 1

```
5 5 1 5
0 1 1 0 0
1 2
1 3
2 5
3 4
4 5
```

Sample Output 1

```
2
```

Explanation. Vertices 1, 4, 5 belong to V_A ; vertices 2, 3 belong to V_B .

- $V^*(5) = 0$.
- Vertex $4 \in V_A$ has only one out-edge $\langle 4, 5 \rangle$, so $V^*(4) = 1 + V^*(5) = 1$.
- Vertex $3 \in V_B$ has only one out-edge $\langle 3, 4 \rangle$, so $V^*(3) = 1 + V^*(4) = 2$.
- Vertex $2 \in V_B$ has only one out-edge $\langle 2, 5 \rangle$, so $V^*(2) = 1 + V^*(5) = 1$.
- At vertex $1 \in V_A$, Alice picks the outgoing edge that minimizes the number of moves to reach t : going through 2 takes $1 + V^*(2) = 2$ moves, going through 3 takes $1 + V^*(3) = 3$ moves, so Alice kicks the ball to 2 and $V^*(1) = 2$.

Sample Input 2

```
3 3 1 3
0 1 0
1 2
2 1
2 3
```

Sample Output 2

```
-1
```

Explanation. Vertices 1 and 3 belong to V_A ; vertex 2 belongs to V_B . Alice at 1 is forced to kick to 2. From 2, Bob can either deflect back to 1 (which causes an infinite cycle $1 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow \dots$) or deflect forward to 3 (which ends the game in one more move). Since Bob wants to delay the ball forever whenever possible, he picks the cycle, so the ball never reaches 3 and the answer is -1 .

Hint

This problem is related to topological sort. Although the course covers only the DFS-based approach, topological sort can also be carried out with a BFS-like procedure that may be more convenient here.